



Ad Soyad: \_\_\_\_\_

Numara: \_\_\_\_\_

Tarih: \_\_\_\_\_

### Genel Hedef

Bu laboratuvarında Bölüm 13'te öğrendiğin optimizasyon ve dağıtım tekniklerini uygulayacaksın. Önce bir oyun döngüsünün profilini çıkarıp darboğazı bulacak, ardından çarpışma testlerini *spatial hashing* ile hızlandıracak, düşmanlara *Finite State Machine* (FSM) ile yapay zekâ verecek ve son olarak oyununu *PyInstaller* ile dağıtılabılır hâle getireceksin. Dört çalışma bittiğinde elinde profile edilebilir, hızlı, akıllı ve paketlenilebilir bir oyun iskeleti olacak.

## 1 Çalışma 1: cProfile ile Profil Çıkarma

İlk işin oyunu hızlandırmadan önce *nereyi* hızlandıracağını bulmak. Tahminle değil, ölçümle çalışacaksın. *cProfile* sana her fonksiyonun toplam süresini, çağrı sayısını ve kümülatif (alt çağrılar dahil) süresini verir. Buradan en pahalı üç fonksiyonu tespit edip optimize edeceksin.

### Örnek Kod – calisma1.py iskeleti

*calisma1.py* dosyasında yapay olarak yavaşlatılmış bir oyun döngüsü var. Görevin profile alıp en yavaş üç fonksiyonu bulmak ve birini hızlandırmak.

```
1 import pygame, cProfile, pstats, math
2
3 GENISLIK, YUKSEKLIK = 800, 600
4
5 def agir_islem(n=2000):
6     """Yapay olarak yavaş --- her karede çağrılır."""
7     toplam = 0.0
8     for i in range(n):
9         toplam += math.sin(i) * math.cos(i)
10    return toplam
11
12 def cizimleri_yap(ekran, daireler):
13     for (x, y, r) in daireler:
14         pygame.draw.circle(ekran, (255, 215, 0), (int(x), int(y)), r)
15
16 def oyun_dongusu(ekran, saat, font):
17     daireler = [(40 + i * 8, 300, 6) for i in range(80)]
18     sure = 0.0
19     while sure < 3.0:
20         dt = saat.tick(60) / 1000.0
21         sure += dt
22         for olay in pygame.event.get():
23             if olay.type == pygame.QUIT: return
24         agir_islem()
25         ekran.fill((20, 30, 40))
26         cizimleri_yap(ekran, daireler)
27         pygame.display.flip()
```

### cProfile Kullanımı

İki yaygın desen vardır: `with cProfile.Profile() as p:` bağlam yöneticisi veya `p = cProfile.Profile(); p.enable() / p.disable()`. Sonra raporu basmak için:

```
1 import pstats
2 istatistik = pstats.Stats(p).sort_stats("cumulative")
3 istatistik.print_stats(10) # ilk 10 fonksiyon
```

“cumulative” sıralama, kendi gövdesi kısa ama içeriden çağrı yapan üst fonksiyonları doğru sıralar.

### Görev 1.1: Profile Sar

Oyun döngüsünü `cProfile.Profile()` ile sar. `p.enable()` ile ana döngüden hemen önce başlat, `p.disable()` ile döngü bittikten sonra durdur. Programın geri kalanını profile dahil etme.

### Görev 1.2: İlk 10 Fonksiyon

`pstats.Stats(p).sort_stats("cumulative").print_stats(10)` ile en yavaş 10 fonksiyonu yazdır. Çıktıyı incele ve dosyanın üst kısmındaki yorum bloğuna en yavaş 3 fonksiyonun adını yaz (örnek: `# 1. agir_islem ...`).

### Görev 1.3: Darboğazı Hızlandır

`agir_islem` fonksiyonunu hızlandır: ya `n` parametresini azalt, ya sonucu `functools.lru_cache` ile ön-belleğe al, ya da döngüyü `numpy` vektörlemesi ile değiştir. Profili tekrar çıkar ve eski/yeni kümülatif süreleri yorumda karşılaştır.

### Bonus 1: Görsel Profil

`snakeviz` kütüphanesini kur (`uv add snakeviz`) ve profili görselleştir: `p.dump_stats("rapor.prof")` ile diske yaz, sonra `snakeviz rapor.prof` komutunu çalıştır. Tarayıcıda renkli grafiği incele ve hangi fonksiyonun *bant genişliği* en geniş, gözlemle.

## 2 Çalışma 2: Spatial Hashing ile Çarpışma Hızlandırma

200 dolaşan dairenin her birinin diğer 199'a karşı kontrol edildiği saf döngü  $O(n^2) = 40\,000$  test demektir. *Spatial hashing* ile dünyayı sabit boyutlu hücrelere bölersin; her cisim sadece kendi hücresindeki komşularıyla test edilir. 60–80 piksel hücre boyutuyla 200 cisim için test sayısı yaklaşık  $200 \times 4 = 800$ 'e düşer.

### Örnek Kod – calisma2.py iskeleti

`calisma2.py` dosyasında `SpatialHash` sınıfının iskeleti verilmiş, `insert` ve `query` metotlarını dolduracaksın.

```
1 from collections import defaultdict
2
3 class SpatialHash:
4     def __init__(self, hucre=64):
5         self.hucre = hucre
6         self.cells = defaultdict(list)    # (cx, cy) -> [obj, obj, ...]
7
8     def clear(self):
9         self.cells.clear()
10
11     def _hucre_indeksi(self, x, y):
12         return (int(x) // self.hucre, int(y) // self.hucre)
13
14     def insert(self, obj):
15         # GÖREV 2.2: obj.rect'in 4 köşesinin hucre indekslerini hesapla,
16         # her hucreye obj'i ekle (set ile tekrarsız)
17         ...
18
19     def query(self, rect):
20         # GÖREV 2.3: rect'in köşegen köşe hücrelerinden geç, her hücredeki
21         # nesneleri sonuc kumesine ekle, listeyi don
22         ...
```

### Hücre Aralığı Hesabı

Bir `rect` birden çok hücreye yayılabilir; sadece sol-üst köşeyi indekslemek yetmez. Hem sol-üst hem sağ-alt köşelerin indekslerini al, `range(cx0, cx1 + 1)` ve `range(cy0, cy1 + 1)` ile bütün hücreleri gez:

```
1 cx0, cy0 = self._hucresindeksi(rect.left, rect.top)
2 cx1, cy1 = self._hucresindeksi(rect.right, rect.bottom)
3 for cx in range(cx0, cx1 + 1):
4     for cy in range(cy0, cy1 + 1):
5         ...
```

### Görev 2.1: Hücre Sözlüğü

`__init__` metoduna `self.cells = defaultdict(list)` satırını ekle. Hücreleri `(cx, cy)` demet anahtarıyla, içerik olarak nesne listesiyle saklayacağız.

### Görev 2.2: insert

`insert(obj)` metodunu yaz: `obj.rect`'in dört köşesinin hücre indekslerini hesapla, kapsadığı tüm hücrelere `obj`'i ekle. İpucu: aynı hücreye iki kez eklememek için `set` ile takip et veya `if obj not in self.cells[(cx, cy)]` kontrolü yap.

### Görev 2.3: query

`query(rect)` metodunu yaz: verilen `rect`'in kapsadığı hücreleri tara, içlerindeki tüm nesneleri `set` ile birleştir, liste olarak döndür. Aynı nesne iki hücrede bulunabilir; bu yüzden `set` kullan.

### Görev 2.4: Brute Force ile Karşılaştır

Ana döngüde önce *kapalı* olan brute force çarpışma kodunu çalıştır ve FPS'i ölç (köşede yazıyor). Sonra `spatial.clear()` + `insert` + `query` bloğuna geçir, FPS'i tekrar ölç. Yorum satırında iki ölçümü kaydet (örnek: # brute=14 FPS, hash=58 FPS).

### Bonus 2: Hücre Boyutu Süpürmesi

Hücre boyutunu 16, 32, 64, 128 olarak değiştir; her birinde FPS'i ölç. Hangi boyut bu cisim sayısı için en iyi sonucu veriyor? Yorumda kısa bir tablo halinde raporla.

## 3 Çalışma 3: FSM ile Akıllı Düşman

Düşmanına bir karakter kazandıralım. Beş durumlu bir *Finite State Machine*: `idle` (durur, etrafa bakar), `patrol` (devriye gezer), `chase` (oyuncuyu kovalar), `attack` (saldırır) ve `cooldown` (saldırıdan sonra dinlenir). Geçişler oyuncu mesafesine ve zamana göre tetiklenir.

### Örnek Kod – calisma3.py iskeleti

`calisma3.py` dosyasında düşman ve oyuncu iskeleti verilmiş. Düşmanın `guncelle` metodundaki durum geçişlerini tamamlayacaksın.

```
1 class Dusman:
2     def __init__(self, x, y, devriye_noktaları):
3         self.x, self.y = x, y
4         self.durum = "idle"
5         self.zamanlayici = 0.0
6         self.devriye = devriye_noktaları
7         self.hedef_idx = 0
8
```

```

9     def mesafe(self, oyuncu):
10         return math.hypot(self.x - oyuncu.x, self.y - oyuncu.y)
11
12     def guncelle(self, dt, oyuncu):
13         self.zamanlayici += dt
14         if self.durum == "idle":
15             # GOREV 3.1: 2 sn sonra patrol'e gec
16             ...
17         elif self.durum == "patrol":
18             # GOREV 3.2: noktalar arasinda dolas, mesafe<200 ise chase
19             ...
20         elif self.durum == "chase":
21             # GOREV 3.3: oyuncuya yaklas, mesafe<50 ise attack
22             ...
23         elif self.durum == "attack":
24             # GOREV 3.4: 0.5 sn animasyon, sonra cooldown
25             ...
26         elif self.durum == "cooldown":
27             # GOREV 3.4: 1.0 sn dinlen, sonra chase'e don
28             ...

```

### Durum Değişikliği Deseni

Her durum geçişinde `self.zamanlayici` sıfırlanmalı; aksi takdirde yeni durumun zaman tabanlı kontrolü hatalı çalışır. Küçük bir yardımcı yaz:

```

1 def _gecis(self, yeni_durum):
2     self.durum = yeni_durum
3     self.zamanlayici = 0.0

```

Ayrıca `patrol` durumunda hedefe yaklaşmak için yön vektörünü `math.atan2` veya  $(dx, dy) / mesafe$  ile normalleştir.

### Görev 3.1: idle → patrol

`idle` durumunda 2 saniye bekle. `self.zamanlayici >= 2.0` olunca `patrol` durumuna geç ve zamanlayıcıyı sıfırla. Bu süre boyunca düşman hareket etmez.

### Görev 3.2: patrol → chase

`patrol` durumunda `self.devriye[self.hedef_idx]` noktasına doğru `60 px/sn` hızla yürü. Hedefe 5 piksel yaklaşıncaya `hedef_idx` değerini bir artır (modulo). Oyuncuya mesafe 200'den küçükse `chase` durumuna geç.

### Görev 3.3: chase → attack

`chase` durumunda oyuncuya doğru `120 px/sn` hızla git. Mesafe 50'den küçükse `attack` durumuna, mesafe 300'den büyükse `patrol` durumuna geri dön (oyuncu kaçtı).

### Görev 3.4: attack ve cooldown

`attack` durumunda 0.5 saniye boyunca düşmanı sabit tut, çevresine kırmızı bir halka çiz (`ciz` metodu zaten hazır). 0.5 saniye dolunca `cooldown`'a geç. `cooldown` durumunda 1.0 saniye hareketsiz dur, sonra `chase` durumuna geri dön.

### Bonus 3: Görüş Açısı

`patrol` durumunda düşmanın oyuncuyu fark etmesi için sadece mesafe değil, *görüş açısı* da kontrol edilsin. Düşmanın hareket yönü vektörü ile oyuncuya olan vektörün arasındaki açı 60 dereceden küçükse oyuncu görülür sayılsın. `math.acos` ve nokta çarpımı kullan.

## 4 Çalışma 4: PyInstaller Uyumlu resource\_path

PyInstaller `-onefile` ile paketlenmiş bir uygulama çalışırken kendi varlıklarını geçici bir klasöre (`sys._MEIPASS`) açar. Eğer kodun `pygame.image.load("assets/oyuncu.png")` gibi göreceli yol kullanıyorsa, exe çalıştığında dosya bulunamaz. Çözüm: çalışma zamanında doğru kök klasörü tespit eden bir `resource_path` fonksiyonu yazmak.

### Örnek Kod – calisma4.py iskeleti

`calisma4.py` dosyasında `assets/` altında yapay bir görsel oluşturulup yüklenmeye çalışılır. `resource_path` fonksiyonunu sen tamamlayacaksın.

```
1 import sys, os
2 import pygame
3
4 def resource_path(rel_path):
5     """Hem geliştirme hem PyInstaller --onefile altında çalışır."""
6     # GOREV 4.2: _MEIPASS varsa onu, yoksa script dizinini kök yap
7     ...
8
9 def main():
10     pygame.init()
11     ekran = pygame.display.set_mode((640, 480))
12     # GOREV 4.3: pygame.image.load çağrısını resource_path ile sarmala
13     yol = "assets/oyuncu.png"
14     yuzey = pygame.image.load(yol).convert_alpha()
15     ...
```

### \_MEIPASS Mantığı

PyInstaller çalıştırma zamanında `sys._MEIPASS` adında bir öznitelik ekler ve oraya bütün veriyi açar. Geliştirme ortamında bu öznitelik yoktur, bu yüzden `getattr(sys, "_MEIPASS", ...)` ile güvenli oku. Geliştirmede ise `os.path.dirname(os.path.abspath(__file__))` script dosyasının bulunduğu klasörü verir.

```
1 def resource_path(rel_path):
2     kok = getattr(sys, "_MEIPASS",
3                   os.path.dirname(os.path.abspath(__file__)))
4     return os.path.join(kok, rel_path)
```

### Görev 4.1: Modül İçer Aktarımları

`calisma4.py` dosyasının üstüne `import sys` ve `import os` ekle. Bunlar `resource_path` için zorunlu.

### Görev 4.2: resource\_path

`resource_path(rel_path)` fonksiyonunu yaz: `sys._MEIPASS` varsa onu kök al, yoksa `__file__` dosyasının dizinini kök al. Sonra `os.path.join(kok, rel_path)` ile mutlak yolu döndür.

### Görev 4.3: Yükleme Çağrılarını Sarmala

Tüm `pygame.image.load(...)` ve `open(...)` çağrılarını `resource_path` ile sarmala. Örnek: `pygame.image.load(resource_path("assets/oyuncu.png"))`.

#### Görev 4.4: PyInstaller ile Paketle

Önce normal şekilde `python calisma4.py` ile çalıştığını gör. Sonra `uv add -dev pyinstaller` ile pyinstaller'ı kur ve şu komutu çalıştır:

```
1 pyinstaller --onefile --windowed \  
2   --add-data "assets:assets" \  
3   calisma4.py
```

Üretilen `dist/calisma4` (Linux/macOS) veya `dist/calisma4.exe` (Windows) dosyasını çift tıklayarak çalıştır. Görsel açılsa görev tamam.

#### Bonus 4: İkon Ekle

`-icon=icon.ico` (Windows) veya `-icon=icon.icns` (macOS) seçeneği ile uygulamana özel bir ikon ekle. Ayrıca `-name OyunumV1` ile çıktı adını özelleştir. Sonuç olarak `dist/OyunumV1` altında ikonlu bir dağıtım dosyası elde et.

## 5 Bonus: Birleştirme

#### Bonus Görev: Tam Optimizasyon Akışı

Dört çalışmanın bilgilerini birleştirip kendi seçtiğin önceki bir lab projesine (Lab 11 platformer veya Lab 12 ZM v4) uygula:

- `cProfile` ile profile al, en yavaş 3 fonksiyonu bul.
- Çarpışma kontrolünü `SpatialHash` ile yeniden yaz; FPS artışını ölç.
- Düşmanlara FSM ekle (en az 3 durum: `idle`, `chase`, `attack`).
- `resource_path` fonksiyonunu ekle, `pyinstaller --onefile --windowed` ile dağıt.
- Final `dist/` klasörünü zip'le, arkadaşına gönder ve bilgisayarında çalıştığını teyit et.

Bu akış, profesyonel bir oyun stüdyosunun *ship* sürecine en yakın küçük ölçekli simülasyondur.